

CS449 Lecture Handout: Input and Files

Dr. Jack Lange

Lecture 2

tar

What tar is for:

Example: bundle this week's lab files `tar -czf /` / `tar -xzf`

C is parsed top down

What “top down” changes when you write a `.c` file:

Compare with Java:

```
int main(void) {  
    foo();  
}  
void foo(void) { }
```

Types of input

Command line Console (`scanf`) Streams
`argv` — what it is:

`argv[0]` is:

Example: run `./cmdargs hello 42`

`scanf` — format string, `&`, and the return value:

Suppose the call is

```
int n;  
int matched = scanf("%d", /* */);
```

Streams

What a stream is (FIFO):

stdin / stdout / stderr:

printf vs fprintf:

```
int printf(const char *format, ...);  
int fprintf(FILE *stream, const char *format, ...);
```

Picture: program, OS buffer, console — mark a flush



The shell and streams

Why the shell can mix files and streams:

What each does:

- `> ls > files.out`
- `>> ls >> all_files.out`
- `< wc < files.out`
- `| ls | grep .c | wc`

Special streams

`/dev/null:`

`/dev/zero:`

`/dev/random:`

Question: How is `/dev/zero` different from `/dev/null`?

File I/O in programs

Who manages console streams vs files you open:

fopen / fclose:

```
FILE *file = fopen(name, /* mode */);  
fclose(file);
```

ASCII vs raw bytes

Character "7" vs integer 7:

Example:

```
int i = 7;  
fprintf("%d\n", i);
```

Memory:

File:

Example:

```
int i = -7;  
fprintf("%d\n", i);
```

Memory:

File:

Picture: memory on the left, file bytes on the right



Explicit file I/O

Why data has to be in memory first:

explicit vs implicit access:

fread / fwrite:

```
int x = 5;
FILE *outfile = fopen(argv[1], "wb");
fwrite(&x, sizeof(x), 1, outfile);
```

memory:

file:

File position

Who remembers the file offset:

Suppose you read 5 bytes, then read again.

Example: `fseek(fstream, 16, SEEK_SET)`

Picture: file as a row of bytes — before a 5-byte read, after it, after that `fseek`



This handout may not cover every topic from the source slides.